Context Enrichment — Methods

Do sensor context labels come out better from a trained ML model or from an LLM? And does giving the LLM real location help — as raw coordinates, or as a named place?

Dataset: **ExtraSensory** (60 users, ~1-minute samples, 51 self-reported context labels; sensors: accelerometer, gyroscope, watch-accelerometer, audio/MFCC, relative location, phone-state). Each sample is mapped to a fixed three-field schema — **location**, **activity**, **companion** — and scored against the user's own self-report. This note describes the exact method for each approach so the comparison is reproducible and fair.

0. Shared controls (kept identical across every approach)

- **Same label schema.** A fixed Pydantic schema (three fields, each a closed vocabulary); no method may invent a dimension.
- **Split by person.** The official ExtraSensory 5-fold cross-validation (train on train-users, score on held-out test-users) — no subject leakage.
- **Same evaluation set.** A stratified sample *e3_eval_sample* (40 samples per location class per fold = 1,530 samples; 1,522 eligible for location) is reused by all approaches.
- **Eligible rows only.** A field is scored only where at least one source label is observed; samples whose whole field is NaN are dropped so undefined gold does not pollute F1.
- **Gold labels.** Multi-label to one label per field by a fixed priority map (LABEL_MAP); when deriving, NaN is treated as 'not that label'.

1. Approach A — Pure ML control (no LLM)

Aspect	Detail
Model	Logistic regression — `SGDClassifier(loss='log_loss', class_weight='balanced', max_iter=25, tol=1e-3, random_state=0)`. Three independent models, one per field.
Features used	All **225 sensor features** (every column except uuid, timestamp, label_source, and the label:* columns). Includes accel/gyro/watch, audio, phone-state, and *relative* location (lat/long range, diameter, speed, altitude). **No absolute coordinates.**
Preprocessing	Fit once per fold on the training users: median imputation (`keep_empty_features=True`) then standardization. Shared by all three heads.
Training filter	Each head trains only on rows eligible for its field; training rows capped at 120,000 (random; class weights handle imbalance).
Protocol	Train per fold on train-users, predict test-users, concatenate the 5 folds into the full-test prediction set (377,346 rows).
Scoring	macro-F1, balanced accuracy, per-class F1, coverage, over-confidence, confusion — computed on eligible rows only.

The ML control runs at its strongest (full 225-feature input) — a strong control, not a strawman.

2. Approach B — LLM enricher (parameters shared by all conditions)

Every LLM condition differs only in a single location cue appended to the prompt; all parameters below are held constant.

Parameter	Value
Model	`gemini-3.1-flash-lite`, via the google-genai SDK.
Decoding	`temperature = 0` (deterministic).

Parameter	Value
Structured output	`response_mime_type='application/json'` + `response_schema=ContextLabel` (Pydantic Literal enums) — the model can only fill the schema, never add a field. Parsed to a typed object.
Reasoning budget	`ThinkingConfig(thinking_budget=0)` (minimal).
Robustness	Up to 4 retries with exponential backoff on transient errors (503/429/500/UNAVAILABLE...); outputs cached by SHA-256(model+prompt) so a prompt is never called twice.
Few-shot	2 examples per location class + 1 per activity class, drawn **only from that fold's train-users** (leakage-free); ~25-26 examples per fold; fixed seed.
Input (sensor summary)	A ~10-descriptor text summary: local time-of-day, phone & wrist motion (accel-std thresholds), rotation (gyro), GPS speed, altitude range, GPS spread (log-diameter), GPS update count, ambient audio, phone-state flags (on-call / wifi / charging).
Time handling	Uses the built-in `discrete:time_of_day` features (already local time) — no timezone inferred from the epoch.

Example sensor summary (the part common to every condition):

```
local time ~ 6-12h; phone still; vehicle speed (max GPS speed 9.2 m/s);
GPS spread log-diameter 2.1; ambient audio quiet; on wifi
```

3. The three location conditions (only the appended cue changes)

The cue is added to **both** the few-shot examples and the query. Absolute coordinates come from a separately-downloaded file, joined to each sample by (uuid, timestamp); samples without GPS get no cue.

Condition	Appended to the prompt	Source
C0 — sensor only (baseline)	*(nothing added)*	—
C1 — raw GPS, no place	`| GPS coordinates: latitude 32.87421, longitude -117.22137`	Raw absolute coordinates (5 decimals)
C2 — named place	`| nearby place (map lookup): Einstein Bros. Bagels, Regents Park Row (OSM: amenity/fast_food)`	Reverse-geocode of the coordinate (OSM Nominatim): place name + OSM category/type

Same sample, differing only at the tail of the prompt:

```
C0: ...ambient audio quiet; on wifi
C1: ...ambient audio quiet; on wifi | GPS coordinates: latitude 32.87421, longitude -117.22137
C2: ...ambient audio quiet; on wifi | nearby place (map lookup): Einstein Bros. Bagels,
     Regents Park Row (OSM: amenity/fast_food)
```

*C2 detail: reverse geocoding uses OSM Nominatim (zoom 18), coordinates rounded to 4 decimals (~11 m) and cached for reproducibility. The map's place name + OSM category are passed verbatim and the **LLM** maps them to the schema — coordinate-to-label rules are never hard-coded (that would make the geocoder, not the LLM, the thing under test).*

4. Held constant vs. varied

	ML	LLM C0	LLM C1	LLM C2
Evaluation set	shared	shared	shared	shared
Split by person, blind to labels	yes	yes	yes	yes
Core input	225 numeric features	sensor text summary	summary	summary

	ML	LLM C0	LLM C1	LLM C2
Location cue added	- (relative only)	-	raw coordinates	place name
Nature	supervised model	LLM, no add	LLM + coords	LLM + geocoder

5. Result at a glance & honest caveat

Location metric	ML	LLM C0	LLM C1	LLM C2
macro-F1 (8 classes)	.344	.248	.307	**.455**
beach recall	—	.00	.01	**.28**

Raw coordinates (C1) barely help and stay below ML; a **named place** (C2) nearly doubles the LLM's location macro-F1 and lifts beach recall from 0. So the LLM's location weakness was a **missing-semantic-signal** problem, not poor reasoning.

Caveat. C2 is an **LLM + geocoder hybrid** — the geocoder names the place, so it does real work; and ML was never given the coordinates (raw location is close to a per-person identifier, which the source paper excluded on purpose). Read C2 as ‘semantic location unlocks the LLM,’ not ‘the LLM beats ML in general.’